Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-22014 PENGEMBANGAN APLIKASI WEB

Modul Belajar 12

Basis Data PostgreSQL

Menyimpan data proyek secara permanen dan memakainya dari FastAPI

Tim Pengajar
Pengembangan Aplikasi Web. Tahun 2026

Komponen	Keterangan
Pertemuan	12 dari 16
Capaian	Sub-CPMK 12.1 dan 12.2, turunan CPMK0501
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P12 - PostgreSQL - Hands-on/`
Yang perlu disiapkan	PostgreSQL 15 atau lebih baru (atau layanan PostgreSQL daring), psql atau DBeaver, venv dengan FastAPI dari P11

Peta belajar

Modul ini mengikuti bagian yang sama dengan slide. Setiap konsep inti ditulis dalam kotak gelap: itulah kalimat yang harus bisa kalian jelaskan ulang.

Bagian	Judul	Yang dipelajari
1	Basis data relasional dan PostgreSQL	Menjelaskan peran basis data relasional di aplikasi web
2	SQL dasar	Menulis query untuk membuat, membaca, mengubah, dan menghapus data
3	Relasi dan JOIN	Menghubungkan tabel dengan foreign key dan menggabungkannya dengan JOIN
4	SQLAlchemy ORM	Memetakan tabel ke class Python dan menjalankan query dengan session
5	FastAPI dan PostgreSQL	Mengganti list Python dengan basis data di endpoint FastAPI
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Tentang modul ini

Modul ini menemani Pertemuan 12. Setiap kotak merah **Tebak dulu** meminta kalian menulis dugaan sebelum membaca kotak hijau **Jawaban** di bawahnya. Kerjakan contoh kode sambil membaca, jangan hanya dibaca.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
12.1	Menerapkan SQL untuk membuat, mengisi, dan menghubungkan tabel di PostgreSQL	Exit ticket soal 1, hands-on, proyek
12.2	Menerapkan SQLAlchemy dan Alembic untuk menyambungkan FastAPI ke PostgreSQL	Exit ticket soal 2, hands-on, proyek

Keduanya menurunkan CPMK0501: mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Masalah pembuka

Server mati, data ikut hilang

API mata kuliah P11 menyimpan data di list Python. Setiap kali server dimuat ulang, mata kuliah yang ditambahkan lewat Next.js hilang.

Di proyek, data pengguna, KRS, dan nilai harus tetap ada walaupun server mati, dan bisa dipakai beberapa server sekaligus.

> **TEBAK DULU**
> Di mana data seharusnya disimpan, dan bagaimana FastAPI membacanya tanpa menulis ulang semua endpoint?

```
DATA: list[dict] = [
    {"kode": "IF25-22014", "sks": 3},
]

@app.post("/matkul", status_code=201)
def tambah(m: MatkulIn):
    DATA.append(m.model_dump())
    return m
# muat ulang server: DATA kembali semula
```

Bagian 1: Basis data relasional dan PostgreSQL

TUJUAN: MENJELASKAN PERAN BASIS DATA RELASIONAL DI APLIKASI WEB

Penyimpan permanen

> **PostgreSQL**
> Sistem basis data relasional sumber terbuka yang berjalan sebagai server terpisah (bawaan port 5432) dan menyimpan data dalam tabel yang saling terhubung.

Ingat peta ekosistem P1: basis data hanya diakses server, tidak pernah langsung oleh browser. FastAPI menjadi satu-satunya pintu; Next.js meminta lewat API.

```
-- tabel: baris dan kolom
CREATE TABLE matkul (
  id SERIAL PRIMARY KEY,
  kode VARCHAR(12) UNIQUE NOT NULL,
  nama TEXT NOT NULL,
  sks INT CHECK (sks BETWEEN 1 AND 6)
);
```

Istilah yang dipakai hari ini

Istilah	Artinya
tabel, baris, kolom	satu jenis data; satu data; satu atribut
primary key	kolom unik penanda setiap baris, misalnya `id`
foreign key	kolom yang menunjuk primary key tabel lain
constraint	aturan di tingkat basis data: `NOT NULL`, `UNIQUE`, `CHECK`
transaksi	sekumpulan perubahan yang berhasil semua atau batal semua (`COMMIT`, `ROLLBACK`)

Ingat

Validasi berlapis: formulir React di browser untuk kenyamanan, Pydantic di FastAPI untuk API, constraint di PostgreSQL sebagai pertahanan terakhir.

Ikuti bersama: basis data pertama

Kerjakan sambil membaca, satu langkah demi satu langkah.

No	Langkah	Perintah atau kode
1	Buat basis data milik kalian.	`createdb krs_NIM`
2	Masuk dengan psql.	`psql -d krs_NIM`
3	Jalankan skema dari folder `starter/`.	`\i schema.sql`
4	Isi data contoh.	`\i seed.sql`
5	Lihat tabel dan isi satu tabel.	`\dt`

YANG HARUS TERLIHAT

`\dt` menampilkan tiga tabel: mahasiswa, matkul, krs. `SELECT * FROM matkul;` menampilkan lima mata kuliah.

Kalau berbeda

password authentication failed: pakai pengguna yang dibuat saat pemasangan, misalnya `psql -U postgres -d krs_NIM`. *createdb tidak dikenal*: tambahkan folder bin PostgreSQL ke PATH, atau buat basis data lewat DBeaver.

Inti Bagian 1

Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.

Bagian 2: SQL dasar

TUJUAN: MENULIS QUERY UNTUK MEMBUAT, MEMBACA, MENGUBAH, DAN MENGHAPUS DATA

Empat perintah utama

```
INSERT INTO matkul (kode, nama, sks)
VALUES ('IF25-22014', 'Pengembangan Aplikasi Web', 3);

SELECT kode, nama FROM matkul
WHERE sks = 3
ORDER BY nama
LIMIT 10;

UPDATE matkul SET sks = 4 WHERE kode = 'IF25-22014';

DELETE FROM matkul WHERE kode = 'IF25-21099';
```

Empat perintah ini setara dengan POST, GET, PUT, dan DELETE di P11.

`INSERT`: teks dalam tanda kutip tunggal.

`SELECT`: pilih kolom, saring dengan `WHERE`, urutkan dengan `ORDER BY`, batasi dengan `LIMIT`. Jawaban misi 2.

`UPDATE` dan `DELETE` tanpa `WHERE` mengenai **semua** baris. Biasakan menulis `WHERE` lebih dulu.

Inti Bagian 2

INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.

Bagian 3: Relasi dan JOIN

TUJUAN: MENGHUBUNGKAN TABEL DENGAN FOREIGN KEY DAN MENGGABUNGKANNYA DENGAN JOIN

Tabel penghubung KRS

```
CREATE TABLE mahasiswa (
  id SERIAL PRIMARY KEY, nim CHAR(9) UNIQUE, nama TEXT
);
CREATE TABLE krs (
  mahasiswa_id INT REFERENCES mahasiswa(id) ON DELETE CASCADE,
  matkul_id INT REFERENCES matkul(id),
  semester INT,
  PRIMARY KEY (mahasiswa_id, matkul_id, semester)
);

SELECT m.nama, k.nama, k.sks
FROM krs
JOIN mahasiswa m ON m.id = krs.mahasiswa_id
JOIN matkul k ON k.id = krs.matkul_id
WHERE krs.semester = 3;
```

Satu mahasiswa mengambil banyak mata kuliah, satu mata kuliah diambil banyak mahasiswa: banyak-ke-banyak.

Tabel `krs` menghubungkan keduanya dengan dua foreign key `REFERENCES`.

`ON DELETE CASCADE`: mahasiswa dihapus, KRS-nya ikut terhapus.

`JOIN ... ON`: gabungkan baris yang kolom penunjuknya cocok. Jawaban misi 3.

Total SKS per mahasiswa

TEBAK DULU

Query mana yang menghasilkan total SKS setiap mahasiswa? (A) `SELECT SUM(sks) FROM matkul`; (B) `... GROUP BY m.nama` dengan `SUM(k.sks)`; (C) `... ORDER BY m.nama`; (D) `... WHERE SUM(k.sks)`

JAWABAN: B

JOIN tiga tabel lalu `GROUP BY` mahasiswa dengan `SUM(k.sks)`. Syarat pada hasil penjumlahan memakai `HAVING`, bukan `WHERE`.

Inti Bagian 3

Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.

Bagian 4: SQLAlchemy ORM

TUJUAN: MEMETAKAN TABEL KE CLASS PYTHON DAN MENJALANKAN QUERY DENGAN SESSION

Tabel sebagai class

ORM (Object Relational Mapping)

Teknik memetakan tabel basis data ke class dan baris ke object, sehingga query ditulis sebagai kode Python dan diterjemahkan menjadi SQL oleh pustaka seperti SQLAlchemy.

OOP P10 bertemu basis data: `Matkul` adalah class, setiap baris adalah object. Query dengan parameter juga mencegah SQL injection.

```
from sqlalchemy import create_engine, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

engine = create_engine("postgresql+psycopg://web:rahasia@localhost:5432/krs")

class Base(DeclarativeBase): pass

class Matkul(Base):
    __tablename__ = "matkul"
    id: Mapped[int] = mapped_column(primary_key=True)
    kode: Mapped[str] = mapped_column(String(12), unique=True)
    nama: Mapped[str]
    sks: Mapped[int]
```

Session: membaca dan menulis

```
from sqlalchemy import select
from sqlalchemy.orm import Session

with Session(engine) as db:
    q = select(Matkul).where(Matkul.sks == 3)
    q = q.order_by(Matkul.nama)
    semua = db.scalars(q).all()
    satu = db.get(Matkul, 1)

    baru = Matkul(kode="IF25-21030",
                  nama="AI", sks=3)
    db.add(baru)
    db.commit() # simpan permanen
    db.refresh(baru) # isi id baru
```

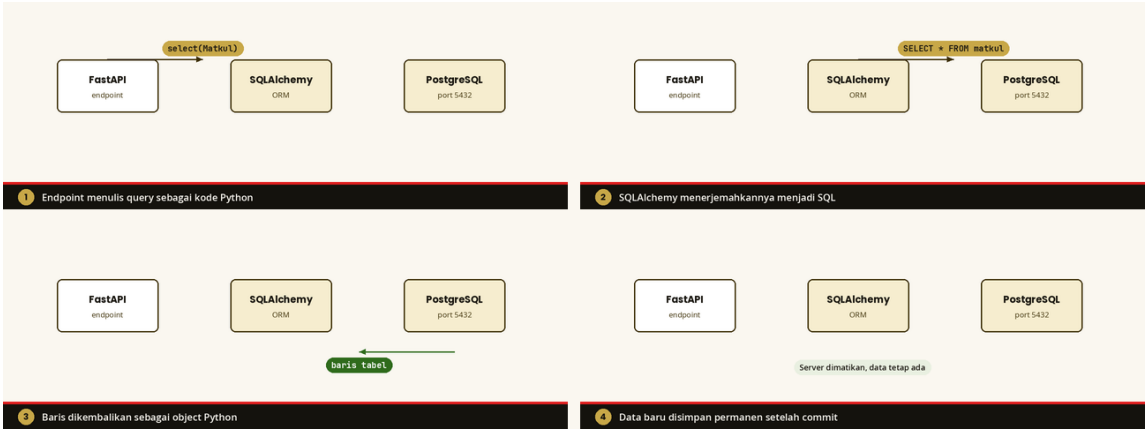
Session: percakapan dengan basis data, sekaligus satu transaksi.

`select`, `where`, dan `order_by`: SELECT, WHERE, ORDER BY dalam bentuk Python.

`db.get(Matkul, 1)`: ambil berdasarkan primary key.

`add` lalu `commit`: INSERT baru tersimpan setelah commit. Tanpa commit, perubahan dibatalkan saat session ditutup.

Dari kode ke SQL



SQLAlchemy menerjemahkan kode Python menjadi SQL dan baris tabel menjadi object; inilah jawaban masalah minggu ini. Versi bergeraknya ada di slide PowerPoint.

Inti Bagian 4

ORM: tabel jadi class, baris jadi object; session menjalankan query dan transaksi.

Bagian 5: FastAPI dan PostgreSQL

TUJUAN: MENGGANTI LIST PYTHON DENGAN BASIS DATA DI ENDPOINT FASTAPI

Endpoint dengan session

```
DB = Annotated[Session, Depends(get_db)] # get_db membuka Session

class MatkulOut(MatkulIn):
    model_config = ConfigDict(from_attributes=True)
    id: int

@app.get("/matkul", response_model=list[MatkulOut])
def daftar(db: DB):
    return db.scalars(select(Matkul)).all()

@app.post("/matkul", response_model=MatkulOut, status_code=201)
def tambah(m: MatkulIn, db: DB):
    baru = Matkul(**m.model_dump())
    db.add(baru); db.commit(); db.refresh(baru)
    return baru
```

Dependency dari P11 kini benar-benar membuka session.

`from_attributes=True`: Pydantic membaca atribut object ORM, bukan hanya dict.

GET: query lalu kembalikan list object; FastAPI mengubahnya menjadi JSON lewat `response_model`.

POST: buat object dari model masukan, simpan, refresh agar `id` terisi. Endpoint lain di Next.js tidak perlu berubah sama sekali.

Inti Bagian 5

Dependency memberi session per request; Pydantic membaca object ORM menjadi JSON.

Bagian 6: Migrasi dan keamanan

TUJUAN: MENGELOLA PERUBAHAN SKEMA DENGAN ALEMBIC DAN MENJAGA KREDENSIAL SERTA QUERY

Alembic

```
pip install alembic
alembic init alembic
# alembic/env.py: target_metadata = Base.metadata

alembic revision --autogenerate -m "tambah kolom semester"
alembic upgrade head # terapkan ke basis data
alembic downgrade -1 # batalkan satu langkah
```

Skema berubah selama proyek: kolom baru, tabel baru. Mengubah tabel manual di laptop sendiri membuat basis data anggota kelompok berbeda-beda.

Alembic membandingkan model SQLAlchemy dengan basis data, lalu menulis berkas migrasi.

Berkas migrasi di-commit ke git; setiap anggota cukup `alembic upgrade head`.

`downgrade` membatalkan migrasi terakhir bila ada yang salah.

Tiga baris yang salah

Yang ditulis	Mengapa salah, dan perbaikannya
query dirakit dengan f-string dari NIM masukan pengguna	SQL injection: masukan bisa berisi perintah SQL. Pakai ORM atau parameter.
password basis data ditulis langsung di `create_engine`	Password ikut ke GitHub. Simpan di `.env` sebagai `DATABASE_URL` dan masukkan `.env` ke `.gitignore`.
`DELETE FROM krs;`	Tanpa WHERE, semua KRS terhapus. Tulis WHERE lebih dulu, uji dengan SELECT yang sama.

Pola kesalahannya

Ketiganya tidak memunculkan error; kerugiannya baru terasa saat data bocor atau hilang.

Giliran kalian

SETENGAH JADI: RELASI DOSEN

Lengkapi: kolom `dosen_id INT _____ dosen(id)` di tabel `matkul`, dan query `SELECT k.nama, d.nama FROM matkul k _____ dosen d ON d.id = k._____;`

MANDIRI: ENDPOINT KRS

Tulis `GET /mahasiswa/{nim}/krs` yang mengembalikan mata kuliah yang diambil mahasiswa itu, 404 bila NIM tidak ada.

JAWABAN

Setengah jadi: REFERENCES, JOIN, dosen_id. Mandiri: cari mahasiswa dengan select where nim, 404 bila None, lalu select Matkul join krs where krs.mahasiswa_id == mhs.id.

Inti Bagian 6

Alembic mencatat perubahan skema; kredensial di .env, query selalu berparameter.

Latihan mandiri

Kerjakan lembar dan berkas awal di folder `02 Hands-on/P12 - PostgreSQL - Hands-on/`.

Tingkat	Waktu	Yang dilatih
Tingkat 1, Dasar	10’	SELECT, WHERE, ORDER BY, INSERT, UPDATE di psql
Tingkat 2, Menengah	15’	JOIN tiga tabel, GROUP BY, HAVING
Tingkat 3, Tantangan	10’	FastAPI dengan SQLAlchemy menggantikan list Python
Pengayaan	sisa	migrasi dengan Alembic

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
UPDATE atau DELETE tanpa WHERE	semua baris berubah	tulis WHERE dulu, uji dengan SELECT
lupa `commit`	perubahan dibatalkan	`db.commit()` setelah `add`
password di kode	bocor ke GitHub	`.env` dan `.gitignore`
SQL dirakit dari masukan	SQL injection	ORM atau parameter
ubah tabel manual	skema anggota kelompok berbeda	migrasi Alembic
object ORM dikembalikan tanpa model	gagal diubah menjadi JSON	`response_model` dengan `from_attributes`

Rangkuman

No	Bagian	Inti
1	Basis data relasional dan PostgreSQL	Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	Relasi dan JOIN	Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.
4	SQLAlchemy ORM	ORM: tabel jadi class, baris jadi object; session menjalankan query dan transaksi.
5	FastAPI dan PostgreSQL	Dependency memberi session per request; Pydantic membaca object ORM menjadi JSON.
6	Migrasi dan keamanan	Alembic mencatat perubahan skema; kredensial di .env, query selalu berparameter.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Mengapa browser tidak boleh langsung mengakses PostgreSQL?
2	Apa beda primary key dan foreign key?
3	Bagaimana memodelkan KRS banyak-ke-banyak?
4	Kapan memakai HAVING?
5	Apa fungsi `db.refresh(baru)`?
6	Mengapa memakai Alembic?

KUNCI CEK PEMAHAMAN

(1) Kredensial dan semua data terbuka; aturan harus diperiksa server. (2) Primary key penanda unik baris; foreign key menunjuk primary key tabel lain. (3) Tabel penghubung krs dengan dua foreign key. (4) Untuk syarat pada hasil agregat setelah GROUP BY. (5) Mengisi atribut dari basis data, misalnya id yang baru dibuat. (6) Perubahan skema tercatat dan bisa diterapkan sama persis di setiap basis data anggota kelompok.

Bacaan lanjut

PostgreSQL, *Tutorial* (`postgresql.org/docs/current/tutorial.html`). SQLAlchemy 2.0, *Unified Tutorial* (`docs.sqlalchemy.org`). Alembic, *Tutorial* (`alembic.sqlalchemy.org`). FastAPI, *SQL (Relational) Databases*.